1. How Sessions Work

Session Flow Steps:

- **Step 1:** User logs in with username and password
- **Step 2:** Server validates the provided credentials
- **Step 3:** Server creates a session and stores it in memory, database, or cache
- **Step 4:** Server sends a session ID to the browser (usually in a cookie)
- **Step 5:** For every subsequent request:
    - Browser automatically sends the session ID cookie
    - Server looks up the session data from storage
    - If session is valid, the user is considered authenticated

Flow:

User Login

↓

Server Creates Session

↓

Session ID = abc123

↓

Browser Stores Cookie

Future Requests:

Cookie: sessionId=abc123

↓

Server checks session store

↓

User authenticated

Problems with Session-Based Authentication

Problem 1: Server Memory and Storage Dependency

- The server must store session data for every active user
- Each logged-in user consumes server memory or database storage

- Example: 1 Million users = 1 Million stored sessions

- As the number of users grows, session storage becomes increasingly expensive
- This creates a linear scaling cost problem

Problem 2: Difficult Horizontal Scaling

- When you have multiple servers behind a load balancer:
    - User logs in through Server A
    - Session is stored only on Server A's memory

    - Next request may reach Server B
    - Server B has no knowledge of the session

    - **Result: Session not found, user authentication fails**

Problem 3: State Management Overhead

- Sessions make the server "stateful"
- **Server must remember for each session:**
    - Session ID
    - User data and profile information
    - Expiration time
    - User permissions and roles
    - Activity status

- **Managing millions of active sessions becomes challenging**
- Requires additional infrastructure for session cleanup and expiration
- Increases server memory footprint

Why JWT Became Popular

What is JWT?

- JWT stands for JSON Web Token
- **It is a "stateless" authentication mechanism**
- **No server-side session storage required**

JWT Workflow:

- **Step 1:** User logs in with credentials
- **Step 2:** Server validates credentials and generates a JWT
- **Step 3:** Server sends the JWT back to the client
- **Step 4:** Client stores the token (localStorage, sessionStorage, or cookie)
- **Step 5:** For every subsequent request:
    - Client sends token in Authorization header: `Bearer <token>`
    - Server verifies the token signature
    - No database lookup required for authentication

JWT Advantages:

Advantage 1: Stateless Authentication

- No session storage required on server
- Server does not need to remember user sessions
- Reduces server memory usage

Advantage 2: Distributed System Ready

- Server A can verify the token
- Server B can verify the token
- Server C can verify the token
- All servers can work independently without sharing session state

Advantage 3: Easier Horizontal Scaling

- Load balancer can send requests to any server
- Any server can verify the JWT signature
- No need for sticky sessions or shared session stores
- Simple and cost-effective scaling

Key:

- Sessions were never "wrong"
- Sessions still work very well for many applications
- Many production systems still use sessions successfully

Why JWT Gained Popularity:

- Simplifies horizontal scaling
- Removes need for centralized session storage
- Works well with distributed systems
- Ideal for Single Page Applications (SPAs)
- Native support for mobile applications
- Perfect for microservice architectures

- **Core Difference:**
- **Sessions:** Server remembers the user (stateful approach)
- **JWT:** Token remembers the user (stateless approach)

When to Use Sessions:

- Traditional server-rendered web applications
- Small to medium scale applications
- When you need immediate session invalidation
- When you have simple scaling requirements

When to Use JWT:

- Modern Single Page Applications (SPAs)
- Mobile applications

- Microservices architectures
- Large scale distributed systems
- When you need stateless authentication
- When horizontal scaling is a priority